DOCUMENTACIÓN DEL PROYECTO

Invitación de boda

Pablo & Marisol · 28 de diciembre de 2026

Next.js 16 · React 19 · TypeScript · Tailwind CSS 4

Guía de funcionamiento, estilos, clases y resolución de dudas

Actualizada el 9 de noviembre de 2026

Contenido

1. El proyecto en una página
2. Cómo está organizado el código
3. Cómo funciona Next.js en este proyecto
4. Cómo funcionan los estilos
5. Catálogo de clases usadas
6. Los componentes, uno por uno
7. Los datos de la boda
8. Imágenes: resolución, carpetas y reemplazo
9. Qué es y para qué sirve PhotoPlaceholder
10. Preguntas frecuentes
11. Errores que ya cometimos y cómo detectarlos
12. Checklist antes de publicar
13. Glosario
14. Próximos pasos

1. El proyecto en una página

Es un sitio de **una sola página** (one-page) que funciona como invitación de boda. El invitado entra, ve la foto principal, la historia de la pareja, la cuenta regresiva, los datos del evento, el código de vestimenta, el hospedaje, la galería y (más adelante) las preguntas frecuentes y la confirmación de asistencia.

Todo el sitio se arma en `app/page.tsx` apilando secciones. Cada sección es un componente independiente con su propio archivo, lo que permite trabajar una a la vez sin romper las demás.

Sección	Componente	Ancla	Estado
Navegación	`components/layout/Header.tsx`	—	Listo
Portada	`components/sections/Hero.tsx`	`#inicio`	Listo
Historia	`components/sections/NuestraHistoria.tsx`	`#historia`	Listo
Cuenta regresiva	`components/sections/CuentaRegresiva.tsx`	`#countdown`	Listo
El gran día	`components/sections/ElGranDia.tsx`	`#gran-dia`	Listo
Vestimenta	`components/sections/CodigoVestimenta.tsx`	`#vestimenta`	Listo
Hospedaje	`components/sections/Hospedaje.tsx`	`#hospedaje`	Listo (sin hoteles aún)
Galería	`components/sections/Galeria.tsx`	`#galeria`	Listo
Preguntas	`components/sections/PreguntasFrecuentes.tsx`	`#faq`	Pendiente
Confirmación	`components/sections/Rsvp.tsx`	`#rsvp`	Pendiente
Pie de página	`components/layout/Footer.tsx`	—	Pendiente

Datos clave del evento

Dato	Valor
Novios	Pablo y Marisol
Fecha	28 de diciembre de 2026
Ceremonia	5:00 PM
Recepción	8:00 PM
Lugar	Centro Cristiano Izcalli
Dirección	Jacarandas 6, Santa Rosa de Lima, 54740 Cuautitlán Izcalli, Méx.
Zona horaria	UTC−6 todo el año

2. Cómo está organizado el código

```
Boda/
├── app/
│   ├── layout.tsx        Envoltura global: fuentes, metadata, <html> y <body>
│   ├── page.tsx          Ensambla todas las secciones en orden
│   └── globals.css       Tokens de color, fuentes y estilos base
├── components/
│   ├── layout/
│   │   ├── Header.tsx    Barra fija + menú hamburguesa
│   │   └── Footer.tsx    Pie de página (pendiente)
│   ├── sections/         Una sección de la página por archivo
│   └── ui/               Piezas reutilizables (Countdown, PhotoPlaceholder)
├── data/
│   └── wedding-data.ts   Fuente única de verdad de los datos
├── docs/                 Esta guía (HTML y PDF)
├── public/
│   ├── images/           Fotos definitivas
│   ├── imagesPrueba/     Fotos de prueba actuales
│   └── icons/            Íconos (flecha)
├── next.config.ts        Configuración de Next
├── tsconfig.json         Configuración de TypeScript
└── package.json          Dependencias y scripts
```

La regla de oro: una sección, un archivo

Cada sección tiene su propio componente. Si algo se rompe en la galería, no afecta al Hero. Si quieres reordenar la página, solo mueves una línea en `app/page.tsx`.

Qué va en cada carpeta

Carpeta	Qué contiene	Qué NO va ahí
`app/`	Rutas, layout y estilos globales	Componentes reutilizables
`components/sections/`	Secciones completas de la página	Lógica de datos
`components/ui/`	Piezas pequeñas reutilizables	Secciones completas
`components/layout/`	Header y Footer	Contenido del evento
`data/`	Textos y datos del evento	Estilos ni JSX
`public/`	Archivos servidos tal cual (fotos, íconos)	Código
`docs/`	Documentación	Código de la app

> **Alias `@/`:** en lugar de escribir `../../../data/wedding-data`, se escribe `@/data/wedding-data`. La raíz del alias es la carpeta del proyecto y se configura en `tsconfig.json`.

3. Cómo funciona Next.js en este proyecto

3.1 App Router: layout y page

Next usa la carpeta `app/` para definir las rutas. Un archivo `page.tsx` es una página; un archivo `layout.tsx` envuelve a todas las páginas que están a su lado y se mantiene montado al navegar.

```
// app/layout.tsx (simplificado)
export default function RootLayout({ children }: LayoutProps<"/">) {
  return (
    <html lang="es">
      <body>{children}</body>
    </html>
  );
}
```

Como este proyecto tiene una sola página, `app/page.tsx` contiene todo. `children` es el contenido que Next inyecta dentro del layout: en este caso, la página.

El tipo `LayoutProps<"/">` es una novedad de Next 16: los tipos de rutas se generan automáticamente, así que no hay que escribir la interfaz de props a mano.

3.2 Server Components y Client Components

En el App Router, **todos los componentes son de servidor por defecto**: se ejecutan en el servidor, no llevan JavaScript al navegador y pueden leer datos directamente. Eso es ideal para una invitación: la mayor parte del sitio es texto e imágenes.

Solo se agrega `"use client"` en la primera línea cuando el componente necesita:

- estado (`useState`),
- efectos (`useEffect`),
- eventos del navegador (`onClick`, `onScroll`),
- APIs del navegador (`window`, `document`, `localStorage`).

En este proyecto solo dos piezas lo necesitan:

Componente	Por qué es cliente
`Header.tsx`	Escucha el scroll y abre/cierra el menú móvil con estado.
`Countdown.tsx`	Actualiza los segundos cada segundo con `setInterval`.

> **Regla práctica:** no pongas `"use client"` "por si acaso". Cada componente cliente suma JavaScript que el invitado debe descargar. Las secciones de contenido no lo necesitan.

3.3 Fuentes con next/font

`next/font/google` descarga las fuentes en tiempo de compilación, las guarda en el propio sitio y genera un CSS sin peticiones a Google. Eso mejora privacidad y evita el salto de texto (FOUT).

```
const cormorant = Cormorant_Garamond({
  subsets: ["latin"],
  style: ["normal", "italic"],
  variable: "--font-cormorant",
  display: "swap",
});
```

Luego la variable se aplica en `<html>` y Tailwind la usa a través de `@theme inline`. Consecuencia importante: **el build necesita internet** para descargar la fuente. En Vercel funciona; en un entorno sin red falla.

3.4 Imágenes con next/image

`next/image` optimiza las fotos: las redimensiona, las convierte a formatos modernos y evita el salto de layout si se le indica el tamaño. En este proyecto se usa con `fill`, que hace que la imagen ocupe todo su contenedor.

```
<div className="relative h-80 w-full overflow-hidden rounded-2xl">
  <Image
    src="/images/novios2.jpg"
    alt="Pablo y Marisol"
    fill
    sizes="(min-width: 768px) 50vw, 100vw"
    className="object-cover"
  />
</div>
```

Requisitos de `fill`:

- El contenedor debe tener `position: relative` (o ser `absolute`), por eso el `relative`.
- El contenedor debe tener altura definida (`h-80`, `aspect-[4/5]`, o estar dentro de un `absolute inset-0`).
- Conviene `overflow-hidden` para recortar las esquinas redondeadas.

3.5 La carpeta public

Todo archivo dentro de `public/` se sirve desde la raíz del sitio. La ruta en el código empieza con `/` y **nunca** incluye la palabra `public`:

Archivo real	Ruta en el código
`public/images/novios1.jpg`	`/images/novios1.jpg`
`public/imagesPrueba/novios1.jpg`	`/imagesPrueba/novios1.jpg`
`public/icons/derecha.png`	`/icons/derecha.png`

> **Mayúsculas y acentos:** en Windows los nombres de archivo no distinguen mayúsculas, pero en Linux (donde corre Vercel) sí. Si en tu equipo funciona `/images/Novios1.jpg` y en producción no, es por esto. Usa minúsculas, sin acentos y sin espacios.

3.6 Metadata

El objeto `metadata` de `app/layout.tsx` define el título de la pestaña y la descripción que aparece al compartir el enlace. Se puede ampliar con una imagen para redes sociales (`opengraph-image`) más adelante.

3.7 Configuración del proyecto

`next.config.ts` activa `cacheComponents`, `partialPrefetching` y la regla de Turbopack para procesar CSS. No hace falta tocarla para el desarrollo normal.

4. Cómo funcionan los estilos

El proyecto usa **Tailwind CSS 4**. La idea central es escribir el estilo directamente en el atributo `className` del elemento, combinando clases pequeñas de una sola responsabilidad.

```
<h2 className="mt-6 font-serif text-4xl text-coffee md:text-5xl">
  Nuestra historia
</h2>
```

Ese `className` equivale a este CSS:

```
h2 {
  margin-top: 1.5rem;
  font-family: var(--font-cormorant);
  font-size: 2.25rem;
  line-height: 2.5rem;
  color: #5c3a21;
}
@media (min-width: 768px) {
  h2 { font-size: 3rem; line-height: 1; }
}
```

4.1 globals.css y la directiva @import

```
@import "tailwindcss";
```

Esa línea carga Tailwind completo: reinicio de estilos, utilidades y variantes. Se importa una sola vez, en `app/layout.tsx`, y aplica a todo el sitio.

4.2 @theme: los tokens del proyecto

```
@theme {
  --color-cream: #f3ebe1;
  --color-sand: #d8bfa8;
  --color-terracotta: #c08d52;
  --color-cinnamon: #a66443;
  --color-coffee: #5c3a21;
  --color-ink: #1a1817;
}
```

Al declarar `--color-terracotta`, Tailwind genera automáticamente todas las utilidades de color derivadas:

Utilidad	Resultado
`bg-terracotta`	fondo terracota
`text-terracotta`	color de texto terracota
`border-terracotta`	borde terracota
`bg-terracotta/80`	fondo terracota al 80 % de opacidad

Por eso en los componentes nunca se escribe `#c08d52`: se escribe `bg-terracotta`. Si un día cambia el tono, se cambia en un solo lugar.

4.3 @theme inline: las fuentes

```
@theme inline {
  --font-sans: var(--font-montserrat);
  --font-serif: var(--font-cormorant);
}
```

Se usa `inline` porque el valor real lo inyecta `next/font` en el elemento `<html>` después de compilar. Sin `inline`, Tailwind intentaría resolver la variable antes de que exista. Resultado: `font-sans` y `font-serif` funcionan.

4.4 @layer base: estilos globales

```
@layer base {
  body {
    background-color: var(--color-cream);
    color: var(--color-ink);
    font-family: var(--font-sans);
  }
}
```

`@layer base` agrupa los estilos de etiquetas (body, html, ::selection, :focus-visible). Es el lugar correcto para valores por defecto, no para estilos de un componente.

4.5 Anatomía de una clase

Una clase de Tailwind se lee de izquierda a derecha: **variantes** → **propiedad** → **valor**.

Clase	Lectura
`md:text-5xl`	Desde 768 px, tamaño de texto 5xl
`hover:bg-cinnamon`	Al pasar el mouse, fondo canela
`text-ink/80`	Color de texto ink al 80 %
`sm:first:border-l-0`	Desde 640 px, al primer hijo, sin borde izquierdo
`[&::-webkit-details-marker]:hidden`	Oculta el triángulo nativo de `<summary>`
`group-open:rotate-90`	Cuando el `<details>` padre está abierto, rota 90°

4.6 Breakpoints (responsive)

Prefijo	Desde	Se usa para
`sm:`	640 px	Teléfonos grandes
`md:`	768 px	Tablet vertical / dos columnas
`lg:`	1024 px	Menú completo, escritorio
`xl:`	1280 px	Pantallas amplias
`2xl:`	1536 px	Pantallas muy grandes

Tailwind es **mobile-first**: lo que escribes sin prefijo aplica a móvil, y los prefijos van sumando reglas hacia arriba. Por eso `text-4xl md:text-5xl` significa "4xl en móvil y 5xl desde tablet".

4.7 Escala de espaciado

La unidad base es `0.25rem` = **4 px**. El número se multiplica por 4.

Clase	Valor	Píxeles
`p-1`	0.25rem	4 px
`p-2`	0.5rem	8 px
`p-3`	0.75rem	12 px
`p-4`	1rem	16 px
`p-6`	1.5rem	24 px
`p-8`	2rem	32 px
`p-12`	3rem	48 px
`p-24`	6rem	96 px

La letra indica el lado: `p` padding, `m` margen, `px` izquierda y derecha, `py` arriba y abajo, `pt` / `pb` / `pl` / `pr` un lado, `gap` separación entre hijos de flex o grid, `space-y` separación vertical entre hijos.

4.8 Valores arbitrarios

Cuando la escala no alcanza, se usan corchetes: `text-[11px]`, `tracking-[0.45em]`, `leading-[0.95]`, `aspect-[4/5]`, `max-h-[calc(100svh-5rem)]`, `grid-cols-[0.8fr_1.2fr]`. Los guiones bajos `_` representan espacios dentro del valor.

4.9 Estados y variantes

Variante	Cuándo se aplica	Ejemplo
`hover:`	Mouse encima	`hover:bg-cinnamon`
`focus-visible:`	Foco por teclado	`focus-visible:outline-2`
`group` + `group-hover:`	Padre con clase group + hover sobre el padre	`group-hover:scale-105`
`group-open:`	Padre `details` abierto	`group-open:rotate-90`
`first:` / `last:`	Primer o último hijo	`first:border-l-0`
`md:` / `lg:`	Breakpoint de ancho	`md:grid-cols-2`

4.10 Transiciones

```
transition-colors duration-300
```

Anima solo las propiedades de color durante 300 ms. En las imágenes se usa `transition-transform duration-500` con `group-hover:scale-105` para el zoom lento.

> **Tailwind falla en silencio.** Si una clase está mal escrita, no hay error de compilación ni de consola: simplemente no se genera CSS. Revisa el inspector del navegador y confirma que la propiedad existe. Los errores más comunes son letras cambiadas (`coffe` en vez de `coffee`) y clases pegadas (`px-8py-10` en vez de `px-8 py-10`).

5. Catálogo de clases usadas

Las clases están agrupadas por función. "Proyecto" indica dónde se usa realmente.

5.1 Estructura y posición

Clase	Qué hace	Proyecto
`fixed`	Fija el elemento a la ventana; no se mueve al hacer scroll	Header
`absolute`	Posiciona respecto al ancestro posicionado más cercano	Hero fondo, overlays
`relative`	Posiciona respecto a su lugar normal y habilita hijos absolutos	Hero, Historia, Galería
`inset-0`	top/right/bottom/left en 0: ocupa todo el contenedor	Fondo del Hero, overlays
`inset-x-0`	Izquierda y derecha en 0	Header
`top-0`	Separación superior de 0	Header
`z-10` / `z-50`	Nivel de apilamiento	Contenido del Hero / Header
`-translate-x-1/2`	Desplaza a la izquierda la mitad de su ancho (centrado con `left-1/2`)	Flecha del Hero
`overflow-hidden`	Recorta lo que se sale del contenedor	Hero, fotos redondeadas
`overflow-y-auto`	Scroll vertical interno si hace falta	Menú móvil
`pointer-events-none`	No intercepta clics	Borde decorativo de la foto

5.2 Flexbox

Clase	Qué hace
`flex`	Convierte el contenedor en flex
`inline-flex`	Flex que se comporta como texto en línea (botones)
`flex-col`	Dirección vertical
`flex-1`	Ocupa el espacio restante
`items-center`	Alinea hijos en el eje transversal (vertical en fila)
`justify-between`	Separa hijos a los extremos
`justify-center`	Centra hijos en el eje principal
`gap-2` / `gap-6`	Separación entre hijos (8 px / 24 px)
`flex-wrap`	Permite que los hijos salten de línea

5.3 Grid

Clase	Qué hace	Proyecto
`grid`	Contenedor de cuadrícula	Varias secciones
`grid-cols-2`	Dos columnas iguales	Galería móvil, Countdown
`md:grid-cols-3`	Tres columnas desde 768 px	Galería
`md:grid-cols-2`	Dos columnas desde 768 px	Historia, Hospedaje
`grid-cols-[0.8fr_1.2fr]`	Columnas con proporción 0.8 / 1.2	Vestimenta
`gap-3` / `gap-12`	Separación entre celdas	Galería / Historia

5.4 Tamaño y ancho

Clase	Qué hace
`w-full`	Ancho 100 % del contenedor
`h-full`	Altura 100 % del contenedor
`h-10 w-10`	40 × 40 px (botón hamburguesa)
`h-20`	80 px de alto (barra del header)
`min-h-svh`	Altura mínima = alto visible del móvil (small viewport)
`max-w-3xl` … `max-w-6xl`	Ancho máximo tipográfico del contenedor
`mx-auto`	Márgenes laterales automáticos: centra el contenedor
`aspect-[4/5]`	Proporción 4:5 (retrato)
`aspect-square`	Proporción 1:1 (no usado aún)

5.5 Tipografía

Clase	Qué hace	Proyecto
`font-serif`	Cormorant Garamond (títulos)	Encabezados
`font-sans`	Montserrat (cuerpo y botones)	Párrafos
`text-xs`	0.75rem / 12 px	Etiquetas
`text-sm`	0.875rem / 14 px	Textos secundarios
`text-base`	1rem / 16 px	Párrafos
`text-lg`	1.125rem / 18 px	Historia
`text-4xl`	2.25rem / 36 px	Títulos de sección
`text-5xl` … `text-9xl`	Escala de títulos grandes	Hero
`text-[11px]`	Tamaño exacto de 11 px	Etiquetas pequeñas
`leading-none` / `leading-[0.95]`	Interlineado ajustado	Nombres del Hero
`leading-relaxed`	Interlineado amplio para lectura	Párrafos
`tracking-[0.45em]`	Espaciado entre letras	Etiquetas en mayúsculas
`uppercase`	Texto en mayúsculas	Etiquetas y botones
`italic`	Cursiva	El "&" del Hero
`text-center` / `md:text-left`	Alineación de texto	Secciones

5.6 Color

Clase	Qué hace
`bg-cream`	Fondo crema
`bg-coffee`	Fondo café (countdown)
`bg-sand/15`	Fondo arena al 15 % (Hospedaje)
`text-ink/80`	Texto principal al 80 %
`text-cream/70`	Texto crema al 70 % sobre fondo oscuro
`border-sand/60`	Borde arena al 60 %
`bg-transparent`	Fondo transparente

5.7 Bordes, radios y efectos

Clase	Qué hace
`border`	Borde de 1 px
`border-b` / `border-t`	Borde solo abajo / arriba
`border-dashed`	Borde discontinuo
`rounded-full`	Borde completamente redondeado (píldora)
`rounded-xl` / `rounded-2xl`	Esquinas redondeadas
`shadow-sm`	Sombra suave
`backdrop-blur-sm`	Difumina lo que hay detrás (header con scroll)
`opacity-60`	Opacidad del 60 % (textura)
`bg-linear-to-b from-ink/70 via-ink/45 to-ink/80`	Degradado vertical sobre la foto del Hero

5.8 Visibilidad responsive

Clase	Qué hace	Proyecto
`hidden`	Oculta el elemento	Menú de escritorio en móvil
`hidden md:flex`	Oculto en móvil, visible y flex desde 768 px	Navegación
`lg:hidden`	Visible hasta 1023 px; oculto en escritorio	Hamburguesa
`hidden lg:flex`	Menú completo desde 1024 px	Header
`block`	Elemento de bloque	Enlaces del menú móvil

6. Los componentes, uno por uno

Header — `components/layout/Header.tsx`

Barra fija arriba. Empieza transparente sobre el Hero y, al pasar 40 px de scroll, cambia a crema con blur. Contiene el monograma "P & M", los enlaces de escritorio y el botón hamburguesa en móvil.

- `"use client"` porque usa `useState` y `useEffect`.
- Un listener de scroll con `{ passive: true }` actualiza `isScrolled`.
- `isSolid = isScrolled || isMenuOpen`: al abrir el menú, el header se vuelve sólido aunque estés arriba.
- El menú móvil bloquea el scroll del fondo y se cierra con `Escape` o al tocar un enlace.
- Los dos `useEffect` limpian sus listeners al desmontar.

Hero — `components/sections/Hero.tsx`

Portada a pantalla completa: foto, degradado, nombres, fecha, hora y ciudad. La foto es la única con `priority` porque es el elemento más grande visible al cargar (LCP). El degradado va encima de la imagen y debajo del texto para garantizar contraste.

NuestraHistoria

Texto, foto y un botón que expande la historia completa. Usa `<details>` y `<summary>` nativos: se abren sin JavaScript, funcionan con teclado y el navegador ya los anuncia como contenido expandible. Las clases `group` y `group-open:` cambian el texto y rotan la flecha.

CuentaRegresiva + Countdown

La sección café envuelve al contador. El contador es cliente: cada segundo calcula la diferencia entre la fecha objetivo y `Date.now()`. No resta uno al valor anterior, así que no se desfasa aunque el navegador esté ocupado.

- Estado inicial `undefined` → muestra `--`.
- Si la fecha ya pasó → muestra un mensaje en lugar de números.
- Al desmontar, `clearInterval` detiene el reloj.

ElGranDia

Una sola tarjeta porque ceremonia y recepción son en el mismo lugar. Muestra nombre, dirección, los dos horarios y el botón "Cómo llegar" con el link de Google Maps. Si `mapsUrl` es `null`, muestra "Ubicación próximamente" en vez de un botón roto.

CodigoVestimenta

Dos columnas: título "Formal" a la izquierda y la descripción en una tarjeta a la derecha. Es contenido estático, sin estado.

Hospedaje

Si el arreglo `hotels` está vacío, muestra un aviso de "muy pronto". Al agregar hoteles, aparecen tarjetas con nombre, nota y enlace. El cambio es automático: no se toca el componente.

Galeria

Grid de 2 columnas en móvil y 3 en escritorio. Si `galleryPhotos` está vacío, pinta 6 placeholders; si tiene fotos, pinta imágenes con `next/image` y zoom al pasar el mouse.

PhotoPlaceholder

Componente propio (no es de Next) que dibuja un recuadro con textura y un texto tenue. Sirve para reservar el espacio de una foto que aún no existe. Se explica en detalle en la sección 9.

7. Los datos de la boda

Todo el contenido variable vive en `data/wedding-data.ts`. Los componentes solo lo importan y lo muestran. Si cambia la fecha o el lugar, se cambia ahí y se actualiza en todas las secciones a la vez.

Exportación	Qué contiene
`couple`	Nombres de los novios
`event`	Fecha ISO, fecha legible, hora y ciudad
`ceremony` / `reception`	Nombre, dirección, hora y link del lugar
`dressCode`	Título y descripción del código de vestimenta
`hotels`	Lista de hoteles (vacía por ahora)
`galleryPhotos`	Lista de fotos de la galería

Por qué la fecha es un string ISO

```
dateIso: "2026-12-28T17:00:00-06:00"
```

El sufijo `-06:00` indica la zona horaria de Cuautitlán Izcalli. Sin él, el navegador interpretaría la hora en la zona del invitado y el contador estaría desfasado para alguien en otro país. Con el offset, todos cuentan al mismo instante.

El patrón mapsUrl: string | null

```
export type Venue = {
  name: string;
  address: string;
  time: string;
  mapsUrl: string | null;
};
```

Mientras no haya link, el valor es `null` y los componentes muestran un texto alternativo. Cuando lo pegues, el botón aparece solo. Es mejor que inventar una URL vacía o romper el enlace.

Cómo cambiar datos

1. Abre `data/wedding-data.ts`.
2. Cambia el valor (fecha, nombre, dirección, hora).
3. Guarda. Next recarga solo.
4. Revisa las secciones afectadas: Hero, Countdown, ElGranDia y el footer a futuro.

> **Si cambias el año:** actualiza `dateIso` (con su offset), `dateLabel` y el `metadata.title` de `app/layout.tsx` si menciona el año. El día de la semana no está escrito a mano: se puede derivar de `dateIso` cuando lo necesites.

8. Imágenes: resolución, carpetas y reemplazo

8.1 Por qué se ven pixeladas

La pixelación no se arregla con CSS. Una imagen tiene un ancho en píxeles **dentro del archivo**. Si el archivo mide 300 px de ancho y se muestra a 1400 px, el navegador estira la información disponible: no puede inventar detalle.

`next/image` optimiza, comprime y elige el tamaño adecuado, pero **nunca supera la resolución del archivo original**. Las fotos de prueba pesan entre 3 y 10 KB, así que su ancho real es pequeño.

> **No existe una clase ni una prop para "poner 1920 px".** Los anchos recomendados son del *archivo de imagen*, no del código. Se definen al exportar la foto desde el celular, la cámara o un editor.

8.2 Dónde se define cada tamaño

Uso	Ancho del archivo	Dónde se define
Hero (pantalla completa)	1920 px o más	Al exportar la foto original
Nuestra Historia	1200 px	Al exportar la foto original
Galería	1000 px	Al exportar cada foto

En el código solo se decide **cómo se muestra**:

```
<!-- El alto lo decide el contenedor -->
<div className="relative h-80 w-full md:h-[400px]">
  <Image ... fill className="object-cover" />
</div>

<!-- La proporción la decide el contenedor -->
<figure className="aspect-[4/5]">
  <Image ... fill className="object-cover" />
</figure>
```

8.3 Cómo saber cuánto mide una foto

- Windows: clic derecho → Propiedades → Detalles → Dimensiones.
- O abre la imagen y mira la barra de título o la información del visor.
- En el navegador: clic derecho sobre la imagen → Inspeccionar; el tamaño natural aparece en las DevTools.

8.4 Cómo exportar bien

1. Parte del archivo original (cámara o celular), no de una copia de WhatsApp: WhatsApp reduce la resolución.
2. Recorta a la proporción que necesitas (horizontal para Hero, vertical para galería).
3. Exporta a **JPG calidad 80–85** o **WebP calidad 80**.
4. Verifica que el ancho sea el de la tabla anterior.
5. Nombra en minúsculas, sin acentos ni espacios: `novios-01.jpg`.

8.5 La prop sizes y la nitidez

`sizes` le dice al navegador cuánto espacio ocupa la imagen en cada breakpoint. Con ese dato, Next genera variantes y sirve la adecuada (incluyendo pantallas 2x). Si `sizes` dice un valor menor al real, el navegador descarga una imagen más pequeña y se ve borrosa aunque el archivo sea grande.

Componente	sizes usado	Por qué
Hero	`100vw`	Ocupa todo el ancho
Historia	`(min-width: 768px) 50vw, 100vw`	Media pantalla en escritorio
Galería	`(min-width: 768px) 33vw, 50vw`	Tres o dos columnas

8.6 Carpetas: imagesPrueba hoy, images después

Mientras pruebas el diseño:

```
public/imagesPrueba/novios1.jpg  →  src="/imagesPrueba/novios1.jpg"
```

Cuando lleguen las fotos definitivas:

```
public/images/novios1.jpg        →  src="/images/novios1.jpg"
```

Dos formas de hacer el cambio:

- **Manual:** mover archivos y actualizar cada `src` (Hero, Historia y las 6 entradas de la galería).
- **Recomendada:** guardar las rutas en `wedding-data.ts` y cambiarlas en un solo archivo.

```
// data/wedding-data.ts
export const photos = {
  hero: "/imagesPrueba/novios1.jpg",
  history: "/imagesPrueba/novios2.jpg",
};

// Hero.tsx
<Image src={photos.hero} ... />
```

Así, el día que cambies de carpeta, editas dos líneas y no tocas los componentes.

8.7 Errores típicos con imágenes

Síntoma	Causa probable
404 en consola	Ruta o nombre incorrecto; recuerda que no lleva `public/`
La imagen no ocupa el contenedor	Falta `fill` y un padre `relative` con altura
Se estira o se deforma	Falta `object-cover`
Se ve borrosa	Archivo pequeño o `sizes` mal calculado
Esquinas no redondeadas	Falta `overflow-hidden` en el contenedor
Funciona en Windows, no en Vercel	Mayúsculas o acentos en el nombre del archivo

9. Qué es y para qué sirve PhotoPlaceholder

`PhotoPlaceholder` es un **componente propio del proyecto**, no una función de Next.js. Dibuja un recuadro con fondo arena, una textura sutil de líneas y un texto pequeño. No carga ninguna imagen.

Sus dos trabajos

1. **Reservar el espacio** con la proporción correcta, para que el diseño se vea completo y no salte cuando llegue la foto.
2. **Hacer visible lo que falta** durante el desarrollo: un recuadro con "Foto de la pareja" se entiende mejor que un hueco vacío.

Cuándo se usa

Situación	Qué usar
No hay foto todavía	`<PhotoPlaceholder />`
Ya hay foto	`<Image />` de `next/image`
Galería vacía	Varios `PhotoPlaceholder` (lo decide el componente)
Galería con fotos	`Image` (lo decide el componente)

Cómo se reemplaza

```
<!-- Antes -->
<PhotoPlaceholder label="Foto de la pareja" className="h-80 w-full rounded-2xl" />

<!-- Después -->
<div className="relative h-80 w-full overflow-hidden rounded-2xl">
  <Image src="/images/novios2.jpg" alt="Pablo y Marisol" fill sizes="50vw" className="object-cover" />
</div>
```

El cambio importante es que las clases de tamaño (`h-80`, `rounded-2xl`) pasan del placeholder al **contenedor** de la imagen, porque `fill` necesita un padre con altura.

> **¿Se borra al final?** No hace falta. Puede quedarse para futuras secciones sin foto (por ejemplo, el Footer) o para el estado vacío de la galería. No suma peso si no se renderiza.

10. Preguntas frecuentes

¿Por qué las fotos se ven pixeladas y dónde pongo el ancho recomendado?

Porque el archivo original tiene pocos píxeles. El ancho (1920 / 1200 / 1000 px) se define al **exportar la foto**, no en el código. En el código solo controlas el tamaño de visualización con clases como `h-80` o `aspect-[4/5]`. Una foto de 300 px estirada a 1400 px siempre se verá borrosa.

¿PhotoPlaceholder solo sirve para cuando no hay imágenes?

Sí. Es un sustituto visual temporal. Cuando existe la foto, se usa `next/image`. Su valor está en reservar el espacio y comunicar qué falta durante el desarrollo.

¿Cuál es la diferencia entre Header y Hero?

El **Header** es la barra de navegación fija (monograma, enlaces, hamburguesa). El **Hero** es la portada a pantalla completa con la foto, los nombres y la fecha. La foto grande va en el Hero, no en el Header.

¿Por qué el header no cambiaba de color?

Porque el Hero ocupaba exactamente el alto de la pantalla y no había más contenido: no existía barra de scroll, así que `window.scrollY` nunca superaba 40. Al agregar la sección de Historia apareció el scroll y el cambio empezó a funcionar.

¿Por qué escribo `&` y no `&`?

En HTML y JSX, el ampersand inicia una entidad. `&` se muestra como `&`. En texto normal de JSX también funciona un `&` suelto, pero `&` es la forma segura y consistente.

¿Necesito `"use client"` en todas las secciones?

No. Solo en las que usan estado, efectos o APIs del navegador. Hoy son dos: Header y Countdown. El resto son componentes de servidor, más livianos.

¿Por qué el contador muestra `--` al principio?

Para que el HTML del servidor y el primer render del cliente coincidan. Si el servidor calculara una hora y el cliente otra, React mostraría un error de hidratación. El `useEffect` calcula el valor real apenas el componente se monta, un instante después.

¿Cómo agrego una foto a la galería?

1. Copia el archivo a `public/images/`.
2. Agrega una entrada a `galleryPhotos` con `src` y `alt`.
3. Listo: el componente deja de mostrar placeholders automáticamente.

¿Cómo cambio la fecha o el lugar?

En `data/wedding-data.ts`. La fecha del contador es `dateIso` (con zona horaria); la legible es `dateLabel`. Los lugares son `ceremony` y `reception`.

¿Por qué una clase no tiene efecto?

Porque está mal escrita y Tailwind la ignora sin avisar. Revisa letra por letra y que no haya dos clases pegadas (`px-8py-10`). En el inspector debe aparecer la propiedad CSS; si no aparece, la clase no existe.

¿Puedo usar CSS normal además de Tailwind?

Sí. `app/globals.css` es CSS normal; ahí están los tokens y los estilos base. Para estilos muy específicos (texturas, animaciones complejas) se puede usar CSS o la prop `style`. Tailwind es la herramienta principal, no una obligación absoluta.

¿Qué pasa si el invitado abre en un celular viejo?

El sitio usa unidades modernas (`svh`), `backdrop-filter` y `next/image`. En navegadores muy antiguos algunos efectos se degradan (por ejemplo, el blur del header), pero el contenido sigue siendo legible. Tailwind 4 requiere navegadores relativamente modernos (Safari 16.4+, Chrome 111+).

11. Errores que ya cometimos y cómo detectarlos

Error	Qué pasaba	Cómo se detecta
`--color-coffe` vs `bg-coffee`	El token tenía una "e" y la clase otra; el fondo no se pintaba	El inspector no muestra `background-color`
`text-cofee`	Clase inexistente; el título usaba el color heredado	El color no coincide con el token
`px-8py-10`	Dos clases pegadas; la tarjeta quedaba sin padding	El texto toca el borde de la tarjeta
`mx-w-15xl`	Clase inventada; el contenedor no tenía ancho máximo	El contenido se estira demasiado en pantallas grandes
`tracking-[o.45em]`	Letra "o" en vez de cero	El espaciado entre letras no cambia
`#galería` en un href	El acento se codifica y no coincide con `id="galeria"`	El enlace no salta a la sección
`suppressContentEditableWarning`	Prop de React equivocada; no silencia la hidratación	Advertencia de hidratación en consola
Sin scroll en la página	El Hero llenaba la pantalla y no había más contenido	El header no cambia al hacer scroll

Método general de diagnóstico

1. Abre DevTools (F12) → pestaña Elements.
2. Selecciona el elemento y revisa "Styles".
3. Si la propiedad no aparece, la clase no se generó: nombre mal escrito o clase inexistente.
4. Si aparece tachada, otra regla la está sobrescribiendo.
5. Mira la consola: los 404 de imágenes y los errores de hidratación salen ahí.

12. Checklist antes de publicar

- Reemplazar todas las fotos de prueba por las definitivas (Hero, Historia, Galería).
- Escribir el texto real de "Nuestra Historia".
- Verificar el link de Google Maps en un celular real.
- Probar en móvil: menú, anclas, contador y galería.
- Revisar que cada imagen tenga un `alt` descriptivo.
- Actualizar `metadata.title` y `description` en `app/layout.tsx`.
- Agregar una imagen para compartir en redes (`opengraph-image`).
- Cambiar el favicon por las iniciales de la pareja.
- Ejecutar `npm run build` sin errores (requiere internet para las fuentes).
- Revisar el sitio en un teléfono real, no solo en DevTools.

13. Glosario

Término	Significado
Componente	Función que devuelve JSX (la descripción de una parte de la interfaz)
Prop	Valor que un componente recibe desde fuera (`label`, `className`)
Estado	Dato que cambia y hace que la interfaz se vuelva a dibujar (`useState`)
Hook	Función especial de React (`useState`, `useEffect`)
Efecto	Código que corre después del render (listeners, intervalos)
Hidratación	Proceso en que React toma el HTML del servidor y lo vuelve interactivo
Server Component	Componente que se ejecuta solo en el servidor y no envía JS
Client Component	Componente marcado con `"use client"` que corre en el navegador
LCP	Largest Contentful Paint: el elemento más grande visible al cargar; se optimiza con `priority`
CLS	Cambio acumulado de layout: los saltos visuales que se evitan reservando espacio
Token	Variable de diseño (color, fuente) declarada una vez y reutilizada
Utility-first	Estilo basado en clases pequeñas de una sola responsabilidad
Breakpoint	Ancho de pantalla a partir del cual cambian los estilos (`md`, `lg`)
Ancla	Enlace a una sección de la misma página (`#historia`)

14. Próximos pasos

1. **Fase 9 — Preguntas frecuentes:** acordeón con `<details>` nativo, sin JavaScript, con datos en `wedding-data.ts`.
2. **Fase 10 — RSVP:** formulario visual primero; después server action y PostgreSQL.
3. **Cierre — Footer:** fondo oscuro con foto y texto arena.
4. **Pulido:** fotos definitivas, texto real, metadata para compartir y favicon.

> **Cómo leer este proyecto si vuelves en semanas:** empieza por `app/page.tsx` para ver el orden de las secciones, sigue con `data/wedding-data.ts` para los datos y luego abre solo la sección que quieras tocar. Todo lo visual está en el `className` del componente correspondiente.